

S3 and CloudFront

Buckets · Objects · Lifecycle · CloudFront CDN · Pre-Signed URLs · Cross-Region Replication

AWS Tutorial · ShopFlow

05-00

Chapter Overview — S3 and CloudFront

S3 (Simple Storage Service) is AWS's object storage — infinitely scalable, 99.999999999% (11 nines) durable, and cheap (\$0.023/GB/month). ShopFlow uses S3 for: product images (500 GB growing at 10 GB/month), customer receipts (PDF documents), application logs, static website assets, and as the data lake for analytics. CloudFront is the CDN (Content Delivery Network) that caches ShopFlow product images at 450+ edge locations worldwide, so a customer in Mumbai sees the image from a nearby edge server, not from us-east-1.

■ S3 and CloudFront Together

S3 is the warehouse — it stores product images durably and cheaply. CloudFront is the delivery network — it copies the most-requested images to edge servers close to customers. Without CloudFront: a customer in Singapore downloads a product image from us-east-1 (100+ ms latency, \$0.09/GB data transfer). With CloudFront: the image is served from the Singapore edge location (5-10 ms latency, \$0.0085/GB). 90% cheaper, 10x faster.

Example: ShopFlow stores "iphone-15-pro-black.jpg" in S3 (10 KB). The first customer from Tokyo loads it — CloudFront fetches from S3 (cache miss). The next 10,000 Tokyo customers load it — CloudFront serves from Tokyo edge (cache hit: zero S3 requests, near-zero latency, \$0.0085/GB vs \$0.09/GB).

Concept	S3	CloudFront
Role	Durable object storage: store files permanently	Content Delivery Network: cache and deliver files globally at low latency
Location	One region (us-east-1 for ShopFlow primary)	450+ edge locations worldwide (Tokyo, Singapore, London, São Paulo, etc.)
Latency	50-150ms from distant regions	5-20ms from nearby edge location
Data transfer cost	\$0.09/GB (egress from S3 to internet)	\$0.0085/GB (egress from CloudFront to internet) — 90% cheaper
Access control	Bucket policies + IAM + ACLs + S3 Block Public Access	Pre-signed URLs/Cookies, WAF integration, geo-restriction, Origin Access Control

05-01

S3 Core Concepts — Buckets, Objects, and Keys

S3 has a flat structure: there are no real folders, only key names that can contain slashes.

"products/iphone-15/images/front.jpg" is not in a folder called "products" — it is an object with that exact key name.

The slash-separated prefix just looks like a folder in the console. Understanding this matters for: access control (prefix-based policies), search queries, and lifecycle rules.

S3 Term	Definition	ShopFlow Example
Bucket	A container for objects. Name globally unique across all AWS accounts in a region.	shopflow-us-east-1-prod-images
Object	A file stored in S3 with a key, value (content), metadata, and version ID.	products/electronics/iphone-15/front.jpg Size: 245 KB Content-Type: image/jpeg
Key	The full path/name of the object (the unique identifier for the object).	products/electronics/iphone-15/front.jpg — this is the complete key; "products" is the prefix.
Prefix	A string that objects share — used for filtering, lifecycle rules, and access control.	products/electronics/* only; that ECS task cannot access anything outside this prefix.
Region	S3 bucket is created in one AWS region; objects can be replicated to other regions.	Created in us-east-1; data residency required for EU customers.
URL	How to access an object publicly (if allowed) or via a pre-signed URL.	https://shopflow-prod-images-prod.s3.us-east-1.amazonaws.com/products/electronics/iphone-15/front.jpg

05-02

Creating an S3 Bucket in the Console

Creating an S3 bucket takes 2 minutes. The most important decision: Block Public Access. For ShopFlow product images, leave Block Public Access ENABLED — customers access images through CloudFront with signed URLs, never directly from S3. Never disable Block Public Access unless you have an explicit need for truly public content.

■ Create ShopFlow Product Images S3 Bucket

1. Search "S3" in the console → Buckets → Create bucket
2. Bucket name: shopflow-product-images-prod-ACCOUNTID (replace ACCOUNTID with your 12-digit AWS account ID)
3. AWS Region: us-east-1
4. Object Ownership: ACLs disabled (Bucket owner enforced) — recommended for new buckets
5. Block Public Access: leave ALL boxes checked (Block all public access = ON)
6. Versioning: Enable (lets you recover accidentally overwritten images)
7. Default encryption: Amazon S3 managed keys (SSE-S3) — enabled by default
8. Click Create bucket — bucket created in seconds

■ aws > S3 > Create bucket

Create bucket

Bucket name

shopflow-product-images-prod-123456789012

■ Must be globally unique. Use your AWS account ID as suffix.

AWS Region

US East (N. Virginia) us-east-1 ■

■ Block Public Access settings for this bucket

■ ■ Block all public access (keep this checked — use CloudFront for delivery)

■ Disabling this makes objects potentially accessible to anyone on the internet. Only disable if you intend to host a truly public website from this bucket.

■ Bucket Versioning

Versioning

■ Disable ■ Enable (recommended — prevents accidental data loss)

■ Default encryption

Encryption type

Server-side encryption with Amazon S3 managed keys (SSE-S3) ■

Create bucket

05-03

S3 in CDK — Complete Bucket Setup

CDK makes S3 bucket configuration explicit and auditable. All settings are in code, reviewed in pull requests, and reproducible across environments. ShopFlow defines all bucket settings in CDK, including lifecycle rules that automatically move old images to cheaper storage tiers.

```
import * as s3 from "aws-cdk-lib/aws-s3";
const productImagesBucket = new s3.Bucket(this, "ProductImages", {
```

```

bucketName: "shopflow-product-images-prod",
versioned: true, // keep old versions for 30 days
blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
publicReadAccess: false,
encryption: s3.BucketEncryption.S3_MANAGED,
enforceSSL: true, // reject HTTP requests; HTTPS only
removalPolicy: RemovalPolicy.RETAIN, // do NOT delete bucket on cdk destroy
autoDeleteObjects: false, // never auto-delete production images
lifecycleRules: [{
  id: "ArchiveOldImages",
  enabled: true,
  transitions: [
    // After 90 days: move to S3 Standard-IA (40% cheaper; same durability)
    {storageClass: s3.StorageClass.INFREQUENT_ACCESS, transitionAfter: Duration.days(90)},
    // After 365 days: move to Glacier Instant Retrieval (68% cheaper)
    {storageClass: s3.StorageClass.GLACIER_INSTANT_RETRIEVAL, transitionAfter: Duration.days(365)},
  ],
  // Delete old versions after 30 days (versioning cleanup)
  noncurrentVersionExpiration: Duration.days(30),
}],
});

```

05-04

Uploading Product Images — Pre-Signed URLs

ShopFlow sellers upload product images directly from their browser to S3. The naive approach: seller uploads to the ShopFlow API server, server uploads to S3 — doubles the bandwidth (seller → API → S3) and adds unnecessary latency. The correct approach: generate a pre-signed URL (a time-limited URL with embedded credentials), return it to the browser, and the browser uploads directly to S3. The API server never touches the image.

■ Generate a Pre-Signed URL for Upload (CLI Demo)

1. `aws s3 presign s3://shopflow-product-images-prod/products/test-product.jpg --expires-in 3600`
2. This generates a URL like: `https://shopflow-product-images-prod.s3.us-east-1.amazonaws.com/products/test-product.jpg?X-Amz-Algorithm=AWS4-HMAC-SHA256&...`
3. The URL is valid for 3600 seconds (1 hour)
4. Share this URL; anyone with it can GET the object (even without AWS credentials)
5. For PUT (upload), use AWS SDK: `s3Presigner.presignPutObject()` in Java

```

// Java 21: generate pre-signed URL for seller to upload directly to S3
@PostMapping("/api/v1/products/{productId}/image-upload-url")
public PresignedUrlResponse generateUploadUrl(@PathVariable String productId) {
    S3Presigner presigner = S3Presigner.create();
    String key = "products/" + productId + "/main.jpg";
    PutObjectPresignRequest presignRequest = PutObjectPresignRequest.builder()
        .signatureDuration(Duration.ofMinutes(15)) // URL valid for 15 minutes
        .putObjectRequest(b -> b
            .bucket("shopflow-product-images-prod")
            .key(key)
            .contentType("image/jpeg")
            .contentLengthRange(1, 10_000_000L) // 1 byte to 10 MB max
        ).build();
    PresignedPutObjectRequest presigned = presigner.presignPutObject(presignRequest);
    return new PresignedUrlResponse(presigned.url().toString(), key);
}
// Frontend: PUT to the presigned URL with the file content directly
// fetch(presignedUrl, {method:"PUT", body: imageFile, headers:{"Content-Type":"image/jpeg"}})

```

CloudFront sits in front of the S3 bucket and caches product images at edge locations worldwide. ShopFlow customers always access images via the CloudFront URL (d1234abcd.cloudfront.net), never directly from S3. CloudFront Origin Access Control (OAC) ensures S3 only accepts requests from CloudFront, never directly from the internet.

■ Create CloudFront Distribution for Product Images

1. Search "CloudFront" in the console → Distributions → Create distribution
2. Origin domain: click the dropdown → select shopflow-product-images-prod bucket
3. Origin access: Origin access control settings (recommended) → Create control setting → Sign requests
4. This generates an S3 bucket policy — click "Copy policy" and save it (apply to S3 next)
5. Default cache behavior: Compress objects automatically: Yes
6. Cache policy: CachingOptimized (long TTL for images that rarely change)
7. Viewer protocol policy: Redirect HTTP to HTTPS
8. Price class: Use all edge locations (best performance) — or Use only North America and Europe to save cost
9. Custom SSL certificate: request or import an ACM certificate for images.shopflow.com
10. Default root object: leave empty (not a website)
11. Click Create distribution — takes 5-10 minutes to deploy globally
12. Go to S3 bucket → Permissions → Bucket policy → paste the policy copied in step 4

■ aws > CloudFront > Create distribution > Origin settings

Create distribution — Origin

Origin domain

shopflow-product-images-prod.s3.us-east-1.amazonaws.com ■

■ Origin access

Origin access

■ Public (not recommended — allows direct S3 access bypassing CloudFront) ■ Origin access control settings (recommended) ■ Legacy access identities

■ Origin access control (OAC) makes S3 private. Only CloudFront can access S3. Customers cannot bypass CloudFront to access S3 directly.

Origin access control

shopflow-images-oac (create new if not exists) ■

■ After creating distribution: copy the S3 bucket policy shown and apply it to the S3 bucket Permissions tab

Next: Default cache behavior

Not all data needs the same storage performance. Product images accessed daily should be in S3 Standard. Images last updated 2 years ago can move to Glacier for 80% savings. ShopFlow lifecycle rules automatically move objects

between storage classes based on age — zero manual effort once configured.

Storage Class	Availability	Retrieval Time	Cost/GB/month	ShopFlow Use Case
S3 Standard	99.99%	Milliseconds (instant)	\$0.023	Active product images, recent receipts, appli
S3 Standard-IA (Infrequent Access)	99.99%	Milliseconds (instant)	\$0.0125	Product images > 90 days old; accessed occ
S3 One Zone-IA	99.5% (single AZ)	Milliseconds	\$0.01	Thumbnails that can be regenerated; derive
S3 Glacier Instant Retrieval	99.9%	Milliseconds	\$0.004	Product images > 1 year old; still needs insta
S3 Glacier Flexible Retrieval	99.9%	Minutes to hours	\$0.0036	Compliance archives; audit logs > 1 year; ba
S3 Glacier Deep Archive	99.9%	12-48 hours	\$0.00099	7-year financial records retention; never acco

05-07

S3 Event Notifications — Trigger Processing on Upload

S3 can trigger a Lambda function, SQS queue, or SNS topic when objects are created, deleted, or transition between storage classes. ShopFlow uses S3 event notifications to: generate thumbnails when a seller uploads a product image, scan uploaded files for malware, and extract metadata for the search index.

```
// S3 event notification in CDK: trigger Lambda when image uploaded
import * as s3n from "aws-cdk-lib/aws-s3-notifications";
// Lambda that generates 3 thumbnail sizes when a product image is uploaded
const thumbnailLambda = new lambda.Function(this, "ThumbnailLambda", {
  runtime: lambda.Runtime.JAVA_21,
  handler: "com.shopflow.ThumbnailHandler::handleRequest",
  code: lambda.Code.fromAsset("thumbnail-lambda/target/thumbnail.jar"),
  memorySize: 1024, // image processing needs RAM
  timeout: Duration.seconds(30),
  environment: {BUCKET: productImagesBucket.bucketName},
});
// Grant Lambda read/write access to the bucket
productImagesBucket.grantReadWrite(thumbnailLambda);
// Trigger Lambda on any .jpg or .png upload to products/ prefix
productImagesBucket.addEventNotification(
  s3.EventType.OBJECT_CREATED,
  new s3n.LambdaDestination(thumbnailLambda),
  {prefix: "products/", suffix: ".jpg"},
);
```

■ Use SQS as the notification destination instead of Lambda directly (bucket → SQS → Lambda). This decouples the upload from the processing, handles Lambda throttling gracefully (SQS buffers messages), and provides retry logic if thumbnail generation fails.

05-08

S3 Replication — Cross-Region for Disaster Recovery

S3 Cross-Region Replication (CRR) automatically copies objects from a source bucket to a destination bucket in a different region. ShopFlow replicates product images from us-east-1 to us-west-2 (DR region) so that if us-east-1 suffers a prolonged outage, the DR site in us-west-2 has all product images ready with zero data loss.

■ Enable Cross-Region Replication for ShopFlow

1. Go to shopflow-product-images-prod bucket → Management tab → Replication rules → Create rule
2. Replication rule name: shopflow-images-crr-to-west2
3. Status: Enabled
4. Source: This bucket (apply to all objects)
5. Destination: Browse S3 → select or create shopflow-product-images-dr-us-west-2 in us-west-2
6. IAM role: Create new role (AWS creates a role allowing S3 to replicate to the destination)
7. Additional replication options: Replicate existing objects (one-time batch job to copy current objects)
8. Click Save — new objects now replicate automatically within minutes
9. Verify: upload a test object → check the DR bucket in us-west-2 Console tab

CRR Scenario	RTO	RPO	ShopFlow Application
Normal operation	N/A	0 — all new uploads replicated within minutes	Product images served from us-east-1 CloudFront; us-west-2 requests may be updated to serve from us-west-2 S3 if us-east-1 S3 is unavailable
us-east-1 partial degradation	Minutes	Near-zero (last few minutes of data may be replicated)	CloudFront may be updated to serve from us-west-2 S3 if us-east-1 S3 is unavailable
us-east-1 complete outage	15-30 minutes (RTO for S3 replication)	Minutes (RPO for S3 replication)	CloudFront serves product images from us-west-2; some recently-uploaded images may be unavailable

05-09

S3 Access Logs and Security Auditing

S3 server access logging records every request made to a bucket: requester identity, bucket, key, operation, response code, bytes transferred. For ShopFlow, this answers: "Did anyone download customer receipt PDFs who should not have?" and "What is the read pattern for product images — which images are most popular?" Enable access logging for all buckets containing sensitive data.

■ Enable S3 Access Logging

1. Go to shopflow-customer-receipts bucket → Properties tab → Server access logging
2. Edit → Enable → Target bucket: shopflow-access-logs-prod (create this bucket first)
3. Log object key prefix: receipts-access-logs/
4. Save changes
5. After 1 hour: check shopflow-access-logs-prod for log files
6. Query logs with Athena: create a table over the log bucket, run SQL to find anomalies

```
-- Athena query: find unauthorized receipt accesses
SELECT requester, key, operation, http_status, request_datetime
FROM shopflow_access_logs
WHERE bucket = "shopflow-customer-receipts"
AND operation = "REST.GET.OBJECT"
AND requester NOT LIKE "arn:aws:sts::*:assumed-role/shopflow-order-svc-role/*"
AND http_status = 200
ORDER BY request_datetime DESC
LIMIT 100;
-- This returns all successful receipt downloads NOT by the order-svc role
-- Every row is a potential security event worth investigating
```

05-10

CloudFront Cache Behaviors — Different Rules for Different Content

CloudFront cache behaviors let you apply different caching rules to different URL patterns. ShopFlow serves both product images (cache for 30 days — images rarely change) and API responses (never cache — always fresh data). A single CloudFront distribution handles both with different cache behaviors per path.

aws > CloudFront > shopflow-distribution > Behaviors

Cache behaviors — path pattern priority

Cache behaviors (evaluated top to bottom; first match wins)

- Path: /api/* → Origin: shopflow-alb → Cache: CachingDisabled → TTL: 0 → Forward headers: All
- Path: /images/* → Origin: shopflow-product-images-prod (S3) → Cache: CachingOptimized → TTL: 2592000 (30 days)
- Path: /static/* → Origin: shopflow-frontend-s3 → Cache: CachingOptimized → TTL: 86400 (1 day)
- Default (*): Origin: shopflow-alb → Cache: CachingDisabled → all unmatched requests go to ALB

Cache headers for /images/* behavior

- Cache-Control: max-age=2592000, immutable — browsers also cache for 30 days
- Vary: Accept — CloudFront caches separate versions for WebP-capable browsers

Create behavior

Cache Behavior	Path	Cache TTL	Reasoning
API calls	api/*	0 (no caching)	Order status, product stock, prices change in real-time; m
Product images	images/*	2,592,000 s (30 days)	Images rarely change; when they do, the URL changes (n
Static assets	static/*	86,400 s (1 day)	JS/CSS bundles change on deployment; cache buster (h
Default	*	0	All other requests to ALB; no caching

05-11 S3 Multipart Upload — Large File Handling

S3 multipart upload is required for objects larger than 5 GB and recommended for any object larger than 100 MB. It splits the file into parts (5 MB to 5 GB each), uploads parts in parallel, then combines them. Benefits: faster uploads (parallel parts), ability to pause and resume, and resilience (only failed parts need to be retried, not the entire file).

Upload Size	Recommended Method	Reason
< 5 MB	Single PUT request	Simple; fast; no overhead
5 MB – 100 MB	Single PUT or multipart (either works)	Multipart only beneficial above 100 MB
100 MB – 5 GB	Multipart upload (recommended)	Parallel parts speed up upload; retry only failed parts
> 5 GB	Multipart upload (required)	S3 maximum single PUT is 5 GB; multipart required for larger objects
> 100 GB	Multipart + S3 Transfer Acceleration	Transfer Acceleration uses CloudFront edge for faster cross-ocean upl

```
// Java 21: multipart upload for large product catalog CSVs
S3TransferManager transferManager = S3TransferManager.builder()
    .s3Client(S3AsyncClient.crtBuilder())
    .minimumPartSizeInBytes(8 * 1024 * 1024L) // 8 MB parts
    .targetThroughputInGbps(10.0)
    .build()
    .build();
FileUpload upload = transferManager.uploadFile(
    UploadFileRequest.builder()
        .source(Path.of("/tmp/product-catalog-full.csv"))
```

```
.putObjectRequest(b -> b
.bucket("shopflow-product-catalog-prod")
.key("catalogs/2024-q4-full.csv")
).build()
);
upload.completionFuture().join(); // blocks until upload complete
// TransferManager handles: part splitting, parallel upload, retry, completion
```

05-12 S3 Intelligent-Tiering — Automatic Cost Optimization

S3 Intelligent-Tiering monitors object access patterns and automatically moves objects between Frequent Access, Infrequent Access, and Archive tiers. No retrieval fees. No manual lifecycle rules needed. ShopFlow uses it for analytics data where access patterns are unpredictable — some old datasets get accessed during year-end audits; Intelligent-Tiering handles this without any configuration changes.

■ Enable S3 Intelligent-Tiering on Analytics Bucket

1. Go to shopflow-analytics-data bucket → Properties tab → Intelligent-Tiering Archive configurations
2. Create configuration → name: shopflow-auto-archive
3. Scope: all objects (or apply to specific prefix: analytics/raw/)
4. Archive access tier: after 90 days without access → move to Archive Access (\$0.0036/GB)
5. Deep Archive access tier: after 180 days → move to Deep Archive (\$0.00099/GB)
6. Click Create — Intelligent-Tiering now manages these objects automatically
7. No changes needed to access patterns — objects retrieved normally; slight delay for archive tiers

Tier	Access Frequency	Cost/GB/month	Retrieval Cost	Retrieval Time
Frequent Access	Accessed recently (within 30 days)	\$0.023	None	Milliseconds (instant)
Infrequent Access	Not accessed in 30-90 days	\$0.0125	None (no retrieval fee!)	Milliseconds
Archive Access	Not accessed in 90+ days (opt-in)	\$0.0036	None	3-5 hours
Deep Archive Access	Not accessed in 180+ days (opt-in)	\$0.00099	None	12-48 hours

05-13 CloudFront Functions and Lambda@Edge

CloudFront Functions and Lambda@Edge let you run code at the edge — in the CloudFront location close to the user — before or after cache lookup. ShopFlow uses CloudFront Functions to: add security headers to all responses, and rewrite image URLs to serve WebP to browsers that support it (40% smaller file size than JPEG).

	CloudFront Functions	Lambda@Edge
Language	JavaScript (subset)	Node.js or Python
Execution location	CloudFront edge (450+ locations)	CloudFront regional edge (13 locations)
Max execution time	1 ms	5 seconds (viewer) / 30 seconds (origin)
Max memory	2 MB	128 MB – 10 GB
Request/response access	Viewer request + viewer response only	All 4 event types: viewer req, origin req, origin resp, viewer resp
Cost	\$0.10 per 1M invocations	\$0.60 per 1M invocations (6x more expensive)
ShopFlow use	Add security headers; URL normalization; A/B testing	Image format negotiation (JPEG → WebP) at origin request; auth token

```
// CloudFront Function: add security headers to all responses
function handler(event) {
```

```
var response = event.response;
var headers = response.headers;
headers["strict-transport-security"] = {value: "max-age=31536000; includeSubdomains; preload"};
headers["x-content-type-options"] = {value: "nosniff"};
headers["x-frame-options"] = {value: "DENY"};
headers["x-xss-protection"] = {value: "1; mode=block"};
headers["referrer-policy"] = {value: "strict-origin-when-cross-origin"};
return response;
}
```

05-14

S3 Object Lock — Immutable Storage for Compliance

S3 Object Lock prevents objects from being deleted or overwritten for a defined period. Required for financial compliance (SOX, PCI DSS): transaction records must be retained for 7 years and must be immutable — even root user cannot delete them. ShopFlow uses Object Lock on the receipts and audit trail buckets.

■ Object Lock Modes

Governance mode: protects objects from deletion by most users but allows privileged IAM users with the s3:BypassGovernanceRetention permission to override retention. Compliance mode: NOBODY can delete or overwrite the object before the retention period expires — not even root, not even AWS support. Compliance mode is required for legal holds and regulated financial data.

Example: ShopFlow sets Compliance mode with 7-year retention on shopflow-customer-receipts. After uploading a receipt PDF: the object is locked for 7 years. Even if an engineer tries to delete it (accidentally or intentionally), S3 returns "Access Denied." The object can only be read, never deleted before 2031.

```
// CDK: Enable Object Lock on receipts bucket (Compliance mode)
const receiptsBucket = new s3.Bucket(this, "CustomerReceipts", {
  bucketName: "shopflow-customer-receipts-prod",
  versioned: true, // Object Lock requires versioning
  objectLockEnabled: true,
  objectLockDefaultRetention: {
    mode: s3.ObjectLockRetentionMode.COMPLIANCE,
    years: 7, // 7-year regulatory retention for financial records
  },
  encryption: s3.BucketEncryption.KMS,
  encryptionKey: receiptKmsKey, // customer-managed key for extra control
  blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
  enforceSSL: true,
});
```

05-15

Complete ShopFlow S3 Architecture

ShopFlow uses 6 S3 buckets, each with a specific purpose, different access patterns, and different storage classes. Here is the complete picture:

Bucket Name	Contents	Storage Class	Access Pattern	Special Config
shopflow-product-images-prod	Product images (JPEG, WebP, PNG)	Standard → IA after 90d	High read (CDN front cache); low write	Object Lock (prevent deletion)
shopflow-customer-receipts-prod	Order receipt PDFs	Standard for 90d → IA forever	Low read (customer download)	Object Lock (Compliance)
shopflow-analytics-data-prod	Kinesis Firehose JSON records	Intelligent Tiering (auto)	Irregular read (analytics jobs)	Object Lock (optional)
shopflow-cloudtrail-logs-prod	CloudTrail API audit logs	Standard → Glacier after 90d	Compliance queries; rare	Object Lock (Governance)
shopflow-access-logs-prod	S3 server access logs (all buckets)	Standard → IA after 30d	Rare (security audit)	Lifecycle: delete after 90 days
shopflow-frontend-prod	React SPA static files (HTML, CSS, JS)	Standard (active)	Very high read via CloudFront	Public, no lock

05-16

S3 Transfer Acceleration — Global Upload Speed

S3 Transfer Acceleration uses CloudFront edge locations to speed up uploads from distant locations. Instead of uploading directly to us-east-1 (which may be slow from Asia or Europe due to public internet routing), the upload goes to the nearest CloudFront edge, which forwards it to S3 via the AWS backbone network (optimized routing, less congestion).

Upload Scenario	Without Transfer Acceleration	With Transfer Acceleration	Speed Gain
Mumbai seller uploads 50MB product catalog to us-east-1	~15 seconds (public internet, variable routing)	~3 seconds (Mumbai edge → AWS backbone → us-east-1)	50% faster
London user uploads profile photo	~8 seconds (transatlantic internet)	~4 seconds (London edge → backbone → us-east-1)	50% faster
US East user uploads same region	~2 seconds	~2 seconds (no benefit; same region)	0% improvement

```
// Enable Transfer Acceleration on the bucket
const catalogBucket = new s3.Bucket(this, "ProductCatalog", {
  transferAcceleration: true,
  versioned: true,
});

// Use the Transfer Acceleration endpoint in the Java SDK
S3Client s3 = S3Client.builder()
  .accelerate(true) // uses bucketname.s3-accelerate.amazonaws.com
  .region(Region.US_EAST_1)
  .build();
```

■ Test whether Transfer Acceleration will help your users at: <https://s3-accelerate-speedtest.s3-accelerate.amazonaws.com/en/accelerate-speed-comparson.html> — AWS provides a real comparison tool. Transfer Acceleration costs \$0.04/GB more than standard upload; only enable if tests show >25% speed improvement for your users.

05-17

S3 Select and Glacier Select — Query Inside Objects

S3 Select lets you run SQL-like queries on individual S3 objects (CSV, JSON, Parquet) and retrieve only matching rows. Instead of downloading a 10 GB CSV to process it in Java, S3 Select filters it server-side and sends only the matching rows. ShopFlow uses S3 Select to query daily sales reports stored in S3 without loading entire files.

```
// S3 Select: extract orders over $500 from a large CSV stored in S3
SelectObjectContentRequest request = SelectObjectContentRequest.builder()
  .bucket("shopflow-analytics-data-prod")
  .key("sales/2024/11/daily-orders.csv.gz")
  .expressionType(ExpressionType.SQL)
  .expression("SELECT * FROM S3Object s WHERE CAST(s.order_total AS FLOAT) > 500.0")
  .inputSerialization(InputSerialization.builder()
    .csv(CSVInput.builder().fileHeaderInfo(FileHeaderInfo.USE).build())
    .compressionType(CompressionType.GZIP)
    .build())
  .outputSerialization(OutputSerialization.builder()
    .csv(CSVOutput.builder().build())
    .build())
  .build();

// Process only matching rows – 10GB file → 50MB result sent over network
// Typical cost: 10GB scanned × $0.002/GB = $0.02; much faster than downloading 10GB
```

05-18

WAF Integration with CloudFront

AWS WAF (Web Application Firewall) integrates with CloudFront to filter malicious HTTP requests at the edge before they reach your origin. ShopFlow uses WAF to block: SQL injection, XSS attacks, known malicious IPs, and rate-limit scrapers that download all product images.

■ Attach WAF to ShopFlow CloudFront Distribution

1. Search "WAF & Shield" in the console → Web ACLs → Create web ACL
2. Resource type: CloudFront distributions (must be created in us-east-1 regardless of CloudFront region)
3. Name: shopflow-cloudfront-waf
4. Add managed rule groups (AWS-managed, regularly updated):
5. - AWSManagedRulesCommonRuleSet (blocks OWASP Top 10)
6. - AWSManagedRulesSQLiRuleSet (blocks SQL injection)
7. - AWSManagedRulesKnownBadInputsRuleSet (blocks known attack payloads)
8. Add custom rule: rate-based rule → 1,000 requests per 5 minutes per IP (blocks scrapers)
9. Associate with CloudFront distribution: shopflow-distribution
10. Click Create — WAF starts filtering all CloudFront traffic

WAF Rule	What It Blocks	False Positive Risk	ShopFlow Config
SQLi detection	Requests containing SQL syntax in query parameters, body, or headers	Low: legitimate requests rarely contain SQL keywords	Enable; blocks SQLi requests weekly in prod
XSS detection	<script> tags and JavaScript event handlers in inputs	Low	Enable; helps prevent stored XSS in product reviews
Rate limiting	IPs making more than N requests per 5 minutes	Medium: real users on shared IP could be blocked	1,000 req/5 min per IP; 100 req/5 min per IP for prod
Geographic blocking	Requests from specific countries (for compliance)	High: intentional blocking	Block countries ShopFlow does not operate in

05-19

Hands-On Lab — Deploy ShopFlow Static Site with S3 and CloudFront

This lab deploys the ShopFlow React frontend to S3 and serves it via CloudFront. You will create an S3 bucket, upload the built React app, create a CloudFront distribution, and configure a custom domain. Estimated time: 20 minutes.

Step	Action	Expected Result
1. Build	Run <code>npm run build</code> (in the shopflow-frontend directory)	build/ folder created with index.html, static/js/*.js, static/css/*.css
2. Create S3 bucket	aws s3 mb s3://shopflow-frontend-prod-ACCOUNTID; region: us-east-1; Block public access: ON	Bucket created
3. Upload build	aws s3 sync build/ s3://shopflow-frontend-prod-ACCOUNTID/	All files uploaded; directory structure preserved in S3 keys
4. Create CloudFront distribution	aws cloudfront create-distribution --s3-bucket shopflow-frontend-prod-ACCOUNTID --domain-name d1234abc.cloudfront.net	Distribution created with domain d1234abc.cloudfront.net
5. Apply S3 OAC policy	aws cloudfront attach-policy --policy arn:aws:cloudfront::us-east-1:policy/CF-Prod-Policy --distribution-id E123456789012	Permissions applied to CloudFront
6. Test	Open d1234abc.cloudfront.net in browser	ShopFlow React app loads; check Network tab: X-Cache: Hit from cloudfront
7. Custom CloudFront page	Error pages → 403 → return index.html with 200 status	403 errors handled gracefully
8. Verify	Open browser DevTools → Network → reload image twice	Images loaded from CloudFront; S3 not hit

05-20

S3 Batch Operations — Process Millions of Objects

S3 Batch Operations lets you run a single operation on billions of objects across one or more S3 buckets. ShopFlow uses Batch Operations for: re-encrypting all objects after a KMS key rotation, running malware scans on all historical uploads, and applying a new metadata tag to all product images. Without Batch Operations, you would write a Lambda that paginates through the bucket — tedious and error-prone.

Batch Operation Type	What It Does	ShopFlow Use Case
Copy	Copy objects to another bucket (or same bucket with new metadata)	Add Content-Type image/webp metadata to 500K product images
Restore from Glacier	Initiate Glacier retrieval for all matching objects	Restore all analytics data from 2022 for a tax audit; retrieve 200 GB
Invoke Lambda	Call a Lambda function for each object (pass bucket/key as input)	Bucket makey-as-input Lambda against all 2M historical product images
Replace ACLs	Update access control lists on all objects	After enabling OAC: ensure all existing objects have ACL cleared
Replicate	Trigger replication for existing objects (one-time backfill)	After enabling Cross-Region Replication: sync all existing 500K objects

■ Run S3 Batch Operations — Re-tag All Product Images

1. S3 → Batch Operations → Create job
2. Region: us-east-1
3. Manifest: S3 Inventory report (set up S3 Inventory first) OR CSV file with bucket,key per line
4. Operation: Copy (to same bucket with new metadata tags)
5. Tagging: Add tag shopflow:scanned=true to all objects
6. Completion report: send to shopflow-batch-ops-reports bucket
7. IAM role: create a role with s3:PutObjectTagging permission on the source bucket
8. Click Create job → Run job → monitor progress in Batch Operations console

05-21

CloudFront Real-Time Logs and Metrics

CloudFront provides two types of logs: standard logs (delivered to S3, 5-10 minute delay) and real-time logs (streamed to Kinesis Data Streams, < 1 second delay). ShopFlow uses real-time logs during Black Friday to monitor cache hit ratios, detect anomalous traffic patterns, and immediately see the impact of WAF rule changes.

Metric	Where To Find It	What It Tells ShopFlow
Cache hit ratio	CloudFront console → Cache statistics	Percentage of requests served from cache vs origin. Target: > 90%
Origin latency	CloudFront console → Origin latency	P50/P90/P99 time for CloudFront to get a response from S3 or ALB
Error rate (4xx, 5xx)	CloudWatch → AWS/CloudFront namespaces	5xx errors indicate origin problems; 4xx indicate bad client requests
Bytes downloaded	CloudFront → Usage reports	Total data transferred via CloudFront. Key for cost tracking (\$0.008/GB)
Requests by edge location	Real-time logs → Kinesis → Kinesis Data Analytics	Which edge locations generate most traffic. Helps decide which edge location to use

05-22

S3 Cost Analysis for ShopFlow

Understanding your S3 cost breakdown prevents bill surprises. At ShopFlow scale (500 GB product images, 100 GB analytics data, 50 GB logs), the monthly S3 + CloudFront bill breaks down as follows:

Cost Item	Volume	Unit Price	Monthly Cost	Optimization
S3 Standard storage	500 GB product images + 50 GB analytics data	\$0.023/GB	\$12.65/month	Lifecycle rules move 300 GB of old images to S3 Standard-IA
S3 Standard-IA (after lifecycle)	300 GB old images (>90 days)	\$0.0125/GB	\$3.75/month	Already optimized via lifecycle rules
S3 PUT requests (product uploads)	50,000 new images/month × 3 sizes	\$0.005/1,000 PUTs	\$0.75/month	Negligible
S3 GET requests (origin fetches by CloudFront)	6M CloudFront misses/month	\$0.0004/1,000 GETs	\$2/month	Improving cache TTL from 1hr to 3hrs
CloudFront data transfer to customers	16 TB/month (product images to browser)	\$0.005/GB (first 10 TB) + \$0.0035/GB	\$87/month	vs \$900/month from S3 directly — CloudFront

Cost Item	Volume	Unit Price	Monthly Cost	Optimization
CloudFront HTTPS requests	50M requests/month	\$0.0100/10,000 requests	\$50/month	WAF rate limiting reduces bot requests
S3 replication to us-west-2	10 GB/month new images replicated	\$0.02/GB transfer	\$0.20/month	Negligible; critical for DR
Total S3 + CloudFront			~\$156/month	vs ~\$926/month without CloudFront

05-23

S3 Inventory and Storage Lens — Visibility at Scale

At 500 GB and millions of objects, you need automated visibility tools. S3 Inventory generates daily or weekly CSV/ORC/Parquet reports of all objects in a bucket (with metadata: size, storage class, replication status, encryption status). S3 Storage Lens gives organization-wide dashboards. ShopFlow uses both to track storage growth, identify unencrypted objects, and find replication failures.

■ Set Up S3 Inventory for Product Images Bucket

1. shopflow-product-images-prod → Management → Inventory configurations → Create
2. Name: shopflow-images-inventory-daily
3. Frequency: Daily
4. Output format: Apache Parquet (queryable with Athena)
5. Destination bucket: shopflow-access-logs-prod (prefix: inventory/)
6. Additional fields: Replication status, Encryption status, Storage class, Size
7. Click Create → first report available within 48 hours
8. Query with Athena: `SELECT storage_class, COUNT(*), SUM(size) FROM inventory GROUP BY storage_class`

S3 Visibility Tool	What It Shows	ShopFlow Query
S3 Inventory (daily)	All objects with metadata: storage class, encryption, replication status	<code>SELECT key FROM inventory WHERE encryption_status = 'Unencrypted'</code>
S3 Storage Lens (org-wide)	Aggregated metrics across all buckets and accounts: total storage, replication status, Replication lag, etc.	<code>Dashboards: Storage Bytes, Data growth, Replication lag, etc.</code>
S3 server access logs	Per-request logs: requester, key, status code, bytes, latency	<code>SELECT key, COUNT(*) FROM access_logs GROUP BY key</code>

05-24

CloudFront Cache Invalidation

When ShopFlow deploys a new version of the React frontend, the old JS and CSS files are still cached in CloudFront edge locations worldwide. Cache invalidation tells CloudFront to evict specific cached objects so the next request fetches fresh content from the origin. Cost: first 1,000 invalidation paths per month are free; \$0.005 per path after that.

■ Invalidate CloudFront Cache After Deployment

1. Go to CloudFront → Distributions → shopflow-distribution → Invalidations tab
2. Click Create invalidation
3. Add object paths: `/static/js/*` (clear all JS bundles) and `/static/css/*` (clear all CSS)
4. Or use `/*` to invalidate everything (use sparingly — invalidates all cached content)
5. Click Create invalidation — propagates to all edge locations within 5-10 minutes
6. Monitor: Invalidation status goes from "In Progress" to "Completed"
7. Verify: open a browser DevTools → Network → reload the page → check that new JS hash is loaded

```
// Automate CloudFront invalidation in CodePipeline after deployment
const invalidation = new codepipeline_actions.LambdaInvokeAction({
  actionName: "InvalidateCloudFront",
  lambda: invalidateLambda, // Lambda that calls cloudfront.createInvalidation()
  runOrder: 3, // run after ECS deploy completes
});
// Lambda code to create invalidation
cloudfront.createInvalidation({
  DistributionId: process.env.CF_DISTRIBUTION_ID,
  InvalidationBatch: {
    CallerReference: Date.now().toString(),
    Paths: {Quantity: 2, Items: ["/static/js/*", "/static/css/*"]},
  }
}).promise();
```

■ Prefer cache busting over invalidation for static assets: include the content hash in the filename (e.g., main.a3f9b2.js). When the file changes, the hash changes, the URL changes, and CloudFront automatically fetches the new file without any invalidation needed. Invalidation is still needed for HTML files (index.html) since you cannot change their URL.

05-25

S3 Object Tagging and Cost Allocation

S3 object tags are key-value pairs attached to individual objects. ShopFlow uses tags for: cost allocation (which team/product is using storage?), lifecycle rule targeting (only archive images tagged product:active=false), and access control (IAM conditions that allow access only to objects tagged with the requester's team).

Tag Key	Tag Value Examples	Use in ShopFlow
shopflow:service	order-svc, product-svc, analytics	Cost allocation: see which microservice generates most S3 storage cost
shopflow:environment	prod, staging, dev	Lifecycle: auto-delete dev uploads after 7 days; keep prod forever
shopflow:content-type	product-image, receipt, catalog, log	Lifecycle: receipts never deleted; logs deleted after 90 days; catalogs never deleted
shopflow:seller-id	seller-12345, seller-67890	Access control: IAM condition allows seller to read/write only objects v
shopflow:processed	true, false	Processing tracking: thumbnail Lambda tags objects processed=true;

```
// IAM policy: seller can only access objects tagged with their seller ID
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::shopflow-product-images-prod/products/*",
  "Condition": {
    "StringEquals": {
      "s3:ExistingObjectTag/shopflow:seller-id": "${cognito-identity.amazonaws.com:sub}"
    }
  }
}
// ${cognito-identity.amazonaws.com:sub} resolves to the seller's Cognito user ID at runtime
```

05-BP

Best Practices and Anti-Patterns

- ✓ Enable S3 Block Public Access at the account level AND bucket level — the account-level setting prevents any future bucket from accidentally being made public, even if a developer forgets to set it at creation
- ✓ Use CloudFront with Origin Access Control (OAC) for all public-facing S3 content — customers get fast global delivery; S3 bucket stays private; content cannot be accessed directly from S3
- ✓ Enable S3 Versioning for all buckets with important data — the storage cost is minimal and it has saved many teams from catastrophic accidental deletes
- ✓ Use pre-signed URLs for user uploads — never route file uploads through your application servers; pre-signed PUT URLs let browsers upload directly to S3, reducing API server load and bandwidth costs by 50-80%

✓ Configure S3 lifecycle rules for cost optimization — define them at bucket creation; old data automatically migrates to cheaper storage classes (Standard → Standard-IA → Glacier) without any manual intervention

✓ Use S3 event notifications with SQS (not direct Lambda) for reliable asynchronous processing — SQS buffers requests, handles throttling, and provides dead-letter queues for failed processing

■ Never disable S3 Block Public Access for buckets containing customer data, financial records, or PII — if you need public access to images, use CloudFront or specific object-level public permissions, not bucket-level public access

■ Do not use S3 for serving content that changes frequently (like API responses or real-time data) — S3 is for static objects; CloudFront cache TTLs mean updates take 5-24 hours to propagate globally; use ElastiCache or DynamoDB for frequently-changing data

05-QR

Quick Reference and Interview Q&A

Question	Answer
What is the difference between S3 Standard and S3 Standard-IA?	S3 Standard: 99.999999999% availability; no minimum storage duration; best for frequently accessed data. S3 Standard-IA: 99.99% availability; minimum 30-day storage duration; best for infrequently accessed data.
How does CloudFront Origin Access S3 Bucket Policy work?	OAC: (1) CloudFront is given an identity (signing certificate). (2) An S3 bucket policy allows CloudFront to access the bucket.
What is a pre-signed URL and how long does it last?	A pre-signed URL contains embedded AWS credentials and a signature that allows the bearer to perform an action on an object. It typically lasts for up to 7 days.
How does S3 achieve 11 nines of durability?	S3 stores each object redundantly across multiple facilities and multiple devices within an AWS region.